Assessment 2

Turing Machine for Sorting Hexadecimal Words

Members:

Luu Minh Thong Tran - 14522561

Minh Hoang Le - 25977873

Thien Bao Minh Dang - 14512584

1 Introduction

This report presents an implementation of a Turing machine that sorts a list of 16-digit hexadecimal words stored on its tape in ascending order. Regarding the algorithm choice, the machine used implements bubble sort where adjacent words are compared and swapped if they are not in the correct order. This process is repeated until the whole list is sorted.

The remaining of the report organised as follow. First it will explain the choice of the the algorithm and the rationale behind it. Then, the documentation will be provided explaining the machine representation and different groups of transitions by the machine used to perform the sorting. It also provides an analysis of the worst-case write complexity, followed by the correctness argument and testing approach to verify the behaviour of the machine.

2 Algorithm choice

2.1 Sorting Strategy

We chose bubble sort a as our algorithm. The algorithm consists of multiple "cycles". During a cycle, the head travels left to right, compares adjacent words, and swaps them if the left one is greater. At the end of a cycle, the rightmost "active" word becomes "inactive" and will be excluded from future cycles. All words are "active" at the start. When there is only one "active" word that remains, the machine restores the punctuation and returns to cell 0.

Algorithm 1 Sorting pseudocode

```
active regions := all words
while active region contains at least two words: do
    compare each adjacent pair from left to right
    if left word > right word then
        Swap two adjacent words
    end if
    exclude the rightmost active word
end while
Restore the list
Return the head to the start and accept
```

2.2 Design rationale and performance measures

This bubble sort approach suits this Turing Machine because all compare-and-swap operations keep the head confined to the two neighbouring words and their delimiters. Reversible digit markers track progress, eliminating the need for a second list, while specialised states remember one digit at a time. This design provides predictable control flow and a clear correctness argument, with one clear trade-off: quadratic complexity. Restoring markers and exchanging digits/words require repeated writes, and every cycle ends by the head returns to cell 0. The machine also performs every shrinking pass even when the list is already sorted. The primary performance measure is the number of tape writes. This also includes transitions that preserve the scanned symbol. Since the head is always confined when comparing and swapping two numbers, no additional storage is needed, and the total tape space stays the same from start to finish.

3 Machine design and documentation

3.1 Tape and machine representation

Each hexadecimal digit has a corresponding temporary marker for comparison and swapping. After swapping, these markers will be cleaned up before advancing. The following table details the marker corresponding to different digits.

Digit	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
Marker	)	!	@	#	$	%	^	&	*	(	a	b	c	d	e	f

The symbols "[" , "," and "]" delimit the list and separate its words. Also, during sorting, the symbol "}" is used to sub-bounded the list because after each pass, the largest element will be at the end of the list. The final "]" symbol that bound the list at the end will remain.

For the representation of the machine rule, a TSV row has five tab-separated fields: current state, current symbol, movement, next symbol, and next state. In the following transformation example, we used `keep` denotes that the scanned tape symbol is preserved and HALT as the halting movement using the character Unicode U+23F9 as well as START for the initial state, which is the character Unicode U+2386. We also use "digit" and "marker" to denote every digit in the alphabet and marker for its corresponding marker.

3.2 Transition groups

There are 847 TSV rows, which contains 3,015 state/symbol transitions. These transitions are categorised into different groups to perform different types of operations to sort the list.

Operation	**Main state families**
Initialisation	`START`; `compare`
Selecting and remembering a digit	`compare`; `compare_X`
Comparison decisions and return paths	`found_compare_X`; `back`; `smaller`; `greater`; `equal`
Restoring digit markers	`restore`; `restore2`; `restore_swap`; `restore_swap2`
Swapping adjacent words	`swap_prep`; `swap_prep2`; `swap`; `swap_X`; `swap_found_X`; `swap_replace_X`; `swap_back_X`; `swap_back`
Advancing to the next pair	`advance`
Completing a pass	`backtrack1`; `backtrack2`
Restoring list punctuation	`clean_up`
Returning and accepting	`return_base`; accepting state ✓

3.3 Group 1 - Initialisation

Transitions in this group:

`(START, "[", →, keep, compare)`

`(compare, "]", ←, keep, return_base)`

These rules move from "[" to the first list position and change to the `compare` state to start comparison. To tackle an empty list, which is just "[]", we use the second rule, which returns to the left if the head reads "]" in the `compare` state. The `return_base` state then accepts at cell 0.

3.4 Group 2 - Selecting and remembering a digit

Transitions in this group:

`(compare, digit, →, marker, compare_digit)`

`(compare_digit, digit, →, keep, compare_digit)`

`(compare_digit, ",", →, keep, found_compare_digit)`

`(compare_digit, "]", ←, keep, backtrack1)`

`(compare_digit, "}", ←, keep, backtrack1)`

`compare` will first write the first digit with its corresponding marker; for example, "A" will be marked as "a" and turn into the `compare_A` state. Then the state `compare_digit` will move to the right until it encounters ",", after which it moves right into the next word and turns into the `found_compare_digit` state. It will be described in Group 3, but the `found_compare_digit` state will move right until it reads a non-marked digit.

There is also an important distinction: if `compare_digit` encounters "]" or "}", it enters `backtrack1` because it has reached the current active boundary of the list.

3.5 Group 3 - Comparison decisions and return paths

Transitions in this group:

`(found_compare_digit, "marker", →, keep, found_compare_digit)`

`(found_compare_digit, "smaller_digit", ←, "smaller_marker", greater)` # Larger

`(found_compare_digit, "digit", ←, "marker", back)` # Equal

`(found_compare_digit, "larger_digit", ←, "larger_marker", smaller)` # Smaller

Here, we used `smaller_digit`, `larger_digit`, `smallermarker` and `larger_marker` to indicate smaller or larger digit compared to the encoded digit in the state.

The `found_compare_digit` state will move to the right until it reads a non-marked digit, where it will perform the comparison. If the first unmarked digit is equal to the memorised digit, then it will move left by turning into the state `back` to compare the next hexadecimal place. If the current digit is smaller than the memorised digit in the state, it will enter the `greater` state and move left to start the swapping path. However, if the current digit is greater than the memorised digit in the state, it will enter the `smaller` state and move left to start restoring without swapping.

3.6 Group 4 - Restoring digit markers

Transitions in this group:

`(restore, marker, →, digit, restore)`

`(restore, digit, →, keep, restore)`

`(restore, ",", →, keep, restore2)`

`(restore2, marker, →, digit, restore2)`

`(restore2, digit, →, keep, restore2)`

`(restore2, ",", ←, keep, advance)`

`(restore2, "]", ←, keep, backtrack1)`

`(restore2, "}", ←, keep, backtrack1)`

Each restoration rule will reverse the marker back to its corresponding hexadecimal digit. The state `restore` will move through the first word and `restore2` will move through the second word. Digits which are already unmarked will be kept unchanged.

After restoring the second word, if `restore2` reads a comma, it means that there is another word inside the active bounded part of the list. Therefore, it will move left and enter `advance` to prepare for the next comparison. If it encounters either the final bracket `]` or the boundary }, it means that the current pass has reached the end of the active part, so it will enter `backtrack1`.

The greater-than path uses another pair of restoration states:

`(restore_swap, marker, →, digit, restore_swap)`

`(restore_swap, digit, →, keep, restore_swap)`

`(restore_swap, ",", →, keep, restore_swap2)`

`(restore_swap2, marker, →, digit, restore_swap2)`

`(restore_swap2, digit, →, keep, restore_swap2)`

`(restore_swap2, ",", ←, keep, swap_prep)`

(restore_swap2, "]", ←, keep, swap_prep)

(restore_swap2, "}", ←, keep, swap_prep)

The states restore_swap and restore_swap2 perform the same restoration. However, after both words are restored, the machine enters swap_prep instead because the first word is greater than the second word and the two words need to be swapped.

3.7 Group 5 - Swapping adjacent words

Transitions in this group:

(swap_prep, digit, ←, keep, swap_prep)

(swap_prep, ",", ←, keep, swap_prep2)

(swap_prep2, digit, ←, keep, swap_prep2)

(swap_prep2, "[", →, keep, swap)

(swap_prep2, ",", →, keep, swap)

The state swap_prep will move left through the second word until it encounters the comma between the two words. It then enters swap_prep2, which moves left through the first word until it reaches the bound before the first word. The machine then moves right and enters swap at the first digit of the first word.

The main swapping transitions are:

(swap, left_digit, →, left_marker, swap_left_digit)

(swap, marker, →, keep, swap)

(swap_left_digit, digit, →, keep, swap_left_digit)

(swap_left_digit, ",", →, keep, swap_found_left_digit)

(swap_found_left_digit, marker, →, keep, swap_found_left_digit)

(swap_found_left_digit, right_digit, ←, left_marker, swap_replace_right_digit)

Here, left_digit and right_digit represent the hexadecimal digits originally read from the left and right words, while left_marker and right_marker are their corresponding marked forms. The digit included in a state name is remembered by the finite control. For example, swap_replace_right_digit remembers the digit taken from the right word while the machine moves left to replace the corresponding position in the first word. These names represent a separate generated transition for each of the 16 hexadecimal digits.

The state swap will find the next unmarked digit in the first word. It replaces this digit with its marker and remembers its value in the state. The state swap_left_digit then moves right to the comma, while swap_found_left_digit skips the positions which have already been processed in the second word.

When it reaches the corresponding unmarked digit in the second word, it writes the marker of the remembered left digit into that position. At the same time, the original right digit is remembered in the state `swap_replace_right_digit`.

The return transitions are:

`(swap_replace_right_digit, marker, ←, keep, swap_replace_right_digit)`

`(swap_replace_right_digit, ",", ←, keep, swap_back_right_digit)`

`(swap_back_right_digit, digit, ←, keep, swap_back_right_digit)`

`(swap_back_right_digit, marker, ←, right_marker, swap_back)`

`(swap_back, marker, ←, keep, swap_back)`

`(swap_back, "[", →, keep, swap)`

`(swap_back, ",", →, keep, swap)`

The machine will move left to the corresponding marked position in the first word and replace it with the marker of the remembered right digit. Therefore, the two digits have been replaced with their marked forms. The state `swap_back` then returns to the beginning of the first word and starts the same process for the next unmarked position.

There is also an important distinction that if `swap` encounters the comma instead of another unmarked digit, all 16 digit positions have already been exchanged:

`(swap, ",", ←, keep, smaller2)`

The machine then enters `smaller2`, moves back to the beginning of the two exchanged words and uses the normal restoration path. This converts all markers back into their corresponding hexadecimal digits and completes the swap.

3.8 Group 6 - Advancing to the next pair

Transitions in this group:

`(advance, digit, ←, keep, advance)`

`(advance, ",", →, keep, compare)`

After the restoration of a pair without reaching the active boundary, `restore2` moves left into the final digit of the second word and enters `advance`. The state `advance` will move left through the second word until it encounters the comma between the two words.

It then moves right and enters `compare`. Therefore, the second word of the previous pair becomes the first word of the next pair. This allows the machine to compare each adjacent pair in the active part of the list and carries the largest element towards the right boundary during each pass.

3.9 Group 7 - Completing a pass

Transitions in this group:

`(backtrack1, digit, ←, keep, backtrack1)`

`(backtrack1, marker, ←, digit, backtrack1)`

`(backtrack1, ",", ←, "}", backtrack2)`

`(backtrack1, "[", →, keep, clean_up)`

The machine enters `backtrack1` after reaching either the final bracket `]` or an existing active boundary }. It will move left through the rightmost word of the active part. If any digit is still marked, `backtrack1` will restore it to its original hexadecimal digit.

When `backtrack1` reaches the comma before this word, it replaces the comma with }. This indicates that the word is now outside the active part of the list. Since each pass moves the greatest active word to this position, the bounded span is shortened by one word after each pass.

The machine then returns to the beginning through the following transitions:

`(backtrack2, digit, ←, keep, backtrack2)`

`(backtrack2, marker, ←, keep, backtrack2)`

`(backtrack2, ",", ←, keep, backtrack2)`

`(backtrack2, "[", →, keep, compare)`

The state `backtrack2` will move left through the remaining active words and commas until it reaches the opening bracket. It then moves right and enters `compare`, which starts the next pass from the first word.

There is also an important distinction that if `backtrack1` reaches the opening bracket directly instead of a comma, only one active word remains. No further sorting pass is required, so the machine enters `clean_up` to restore the list before returning to cell 0 and accepting.

3.10 Group 8 - Restoring the list

Transitions in this group:

`(clean_up, digit, →, keep, clean_up)`

`(clean_up, "}", →, ",", clean_up)`

`(clean_up, "]", ←, keep, return_base)`

After all sorting passes are completed, the state `clean_up` will move right through the whole list. Each } represents a comma which was replaced when the active part of the list was shortened. Therefore, `clean_up` changes every } back into a comma while keeping all the

digits.

After iterations of shortening the bounded active list, the original commas have been changed into }, therefore there should be no original comma remaining inside the list. When `clean_up` reaches the final bracket `]`, it keeps the bracket unchanged, moves left and enters `return_base` where it returns to index 0 and accepts. Therefore, all scratch symbols used as active bounded boundaries are removed and the normal "," of the sorted list is restored.

3.11 Group 9 - Returning and accepting

Transitions in this group:

`(return_base, digit, ←, keep, return_base)`

`(return_base, marker, ←, keep, return_base)`

`(return_base, ",", ←, keep, return_base)`

`(return_base, "}", ←, keep, return_base)`

`(return_base, "]", ←, keep, return_base)`

`(return_base, "[", HALT, keep, ACCEPT)`

The state `return_base` will move left through the restored list while keeping every symbol unchanged. It continues until it reaches the opening bracket `[`, which is located at cell 0.

When the head reads `[`, the machine uses the halt movement and enters the accepting state `ACCEPT`. Therefore, the machine accepts with the head returned to cell 0. At this stage, every marker has been restored to its corresponding digit, every active boundary } has been restored to a comma, and the final tape contains nothing other than the sorted list.

4 Proof of worst case complexity

Let m be the tape length and n be the number of words in the list. Because each word has 16 symbols, with $n-1$ separating commas and two bounding brackets,

$$m = 16n + (n-1) + 2 = 17n + 1 \Rightarrow n = \frac{m-1}{17}$$

Consider a list with n hexadecimal words $[\text{hex}_1, \text{hex}_2, \ldots, \text{hex}_n]$.

Each compare, swap and advance operation only costs at most a constant A writes. This is because it operates only on two adjacent words of fixed length and does not depend on the size of the list nor size of the tape.

Each backtrack will run through the whole active list and visit each symbols at most 1 time. Hence it will takes $Bk + C$ writes for k active words. And for the clean up and return operation it will also visit all the words in the list and hence it requires $Bn + C$ writes.

For an active bounded list of k word, the machine will perform at most $k-1$ operations of compare with advance and possibly swap. After each pass, the active region shrinks by one word. $(k \leftarrow k-1)$

Hence, let $W(m)$ to be the number of write operation with a tape size m, worst case the machine will perform is:

$$\begin{aligned}
W(m) &\leq Bn + C + \sum_{k=2}^{n} (A(k-1) + Bk + C) \\
&= A\left(\frac{n(n-1)}{2}\right) + B\left(\frac{n(n+1)}{2} - 1\right) + C(n-1) + Bn + C \\
&= A\left(\frac{n^2-n}{2}\right) + B\left(\frac{n^2+n-2}{2}\right) + Cn - C + Bn + C \\
&= n^2\left(\frac{A}{2} + \frac{B}{2}\right) + n\left(-\frac{A}{2} + \frac{3B}{2} + C\right) - B \\
&\leq \left(\frac{A}{2} + 2B + C\right) n^2 \qquad \forall\, n \geq 1
\end{aligned}$$

Hence,

$$W(m) = O(n^2) \quad \textbf{(1)}$$

Now consider a tape of a repeated identical list

$$[\text{hex}_1, \ldots, \text{hex}_n] \qquad \text{hex}_1 = \cdots = \text{hex}_n$$

For each pass, it does not perform swapping as the pairs are already sorted (equal). This means that only comparison, restoration and advance are performed, with the same number of operations. When the active bounded list contains k elements, it requires exactly $k-1$ comparisons. Because the list only contains identical words, it takes at least a constant $A' > 0$ writes to perform one comparison and restoration. The backtrack through the active bounded list, on the other hand, requires at least $B'k + C'$ writes for constants $B' \geq 0, C' \geq 0$ because each word of the active bounded list must be visited at least once.

Hence, it requires at least

$$\begin{aligned}
W(m) &\geq \sum_{k=2}^{n} (A'(k-1) + B'k + C') \\
&= A'\left(\frac{n(n-1)}{2}\right) + B'\left(\frac{n(n+1)}{2} - 1\right) + C'(n-1) \\
&= A'\left(\frac{n^2-n}{2}\right) + B'\left(\frac{n^2+n-2}{2}\right) + C'n - C' \\
&= n^2\left(\frac{A'}{2} + \frac{B'}{2}\right) + n\left(-\frac{A'}{2} + \frac{B'}{2} + C'\right) - (B' + C') \\
&\geq A'\frac{n(n-1)}{2} \\
&\geq \frac{A'}{4}n^2 \qquad \forall\, n \geq 2
\end{aligned}$$

Hence,

$$W(m) = \Omega(n^2) \quad \textbf{(2)}$$

(1) and (2) implies $W(m) = \Theta(n^2)$. So, the worst case write complexity of a tape with n hexadecimal word is $\Theta(n^2)$

Additionally, since $n = (m-1)/17$,

$$\begin{aligned}
W(m) &= \Theta(n^2) \\
&= \Theta\left(\left(\frac{m-1}{17}\right)^2\right) \\
&= \Theta\left(\frac{1}{17^2}m^2 - \frac{2}{17^2}m + \frac{1}{17^2}\right) \\
&= \Theta(m^2)
\end{aligned}$$

Therefore, the worst case write complexity both of a tape length m is $\Theta(m^2)$

5 Correctness of the machine

5.1 Correctness of comparison and swapping

Let x and y be adjacent, unmarked active words. The current state is `compare` and the head is currently at the first character of x.

- **Comparison:** The machine performs the swapping if and only if $x > y$ and both words are restored after the comparison/swapping. *Proof.* Before round i, all the prior positions of both words are marked and equal, and all the later positions are unmarked. In round i, the state `compare` marks x_i and the machine changes its state to `compare_`x_i. The machine then moves right to the comma and finds the first unmarked character y_i, which it also marks. If $x_i = y_i$, the state `back` guides the head back to the beginning and starts round $i + 1$, preserving the invariant. If $y_i < x_i$ or $y_i > x_i$, i is the first difference, thus making the state change to `greater` (swap) or `smaller` (no swap) correct. If $x_i = y_i$ for all 16 i, the state is now `equal` and the machine does not swap. Finally, the machine performs a sweep through all the characters and restores them.
- **Swapping:** The swap ends with y in the first word's position and x in the second, with both unmarked. *Proof.* Considering the swap at position i (or $S(i)$). For $j \leq i$, the first word holds the marked character of y_j and the second holds the marked character of x_j. Later positions are unchanged. In each round, the state `swap` marks the first unmarked digit x_{i+1} and remembers it. `swap_found_`x_{i+1} skips i markers to reach y_{i+1}, writes the marked variant of x_{i+1} there and remembers y_{i+1}. Moving left, the machine stops at the rightmost marker of the first word, which is position $i + 1$, and writes the marked character of y_{i+1}. After 16 rounds, the state `swap` reads the comma and the machine starts the restoration process.

Each compare-and-swap step is shown to replace the pair by $(\min(x, y), \max(x, y))$ and does not change anything else.

5.2 Correctness and termination of sorting

During sorting, when $m - k > 0$, the tape has the form $[a_1, ..., a_k\}b_1\}...b_{m-k}]$, with k active words a_i and $m - k$ inactive words b_j. Initially, when $k = m$, the tape has the form $[a_1, ..., a_m]$.

When $m - k > 0$, it can be shown that $b_1 \leq b_2 \leq ... \leq b_{m-k}$ and every $a_i \leq b_1$. When there are no inactive words, this condition is vacuously true.

After $k - 1$ compare-and-swap steps, a_k holds $\max(a_1, ...a_k)$. The state `restore2` then reads } or] and enters `backtrack1`. The machine then finds the comma before a_k and overwrites it with } which makes a_k the new b_1. The active region size is now reduced by one. Since a_k is the maximum active word, every remaining active word is at most the new b_1. The new b_1 is also at most the old b_1, so the invariant is preserved. `backtrack2` returns to [and starts the next pass.

After $m - 1$ passes, the active region size is now one and the list is sorted. On the next comparison attempt, the machine reaches } or] without finding a second active word and enters `backtrack1`. `backtrack1` now reaches [directly, starting the cleanup. This turns every } back to a comma. The head then moves to cell 0 and accepts.

For the empty list, `compare` immediately reads], enters `return_base`, returns to cell 0 and accepts.

5.3 Testing

Our Turing Machine using the TSV file has been thoroughly tested to ensure that it successfully implements the design above. A test passes only if the machine accepts, the head is on cell 0 and the final tape matches the Python sorted result. Python string sort gives the correct result because `0-9` precedes `A-F` in ASCII and all words have the same length.

For testing, we have generated 7 test-case families. The first case is a completely random list of a list size m. An example tape for this case is `[FAF7514CE088A166,483EA4CD23526E91, 204AD4A01C92ABCE]`. Ten random lists were generated for each size from 1 to 49.

We then tested a sorted list, for which the machine should return the exact same tape. An example for this family is `[219F81A7541D8802,32B4447CBC29E135]`.

We also tested reverse-sorted lists. These cases require the machine to perform the largest possible number of adjacent swaps.

Next, we tested possible duplication of words in the tape. For example, one of the tapes in this family was `[EA1D2CABC86440ED,EA1D2CABC86440ED]`. This checks that equal words are preserved and are not unnecessarily swapped.

We also tested the empty list, which is the tape `[]`. This checks the special path that returns the head directly to cell 0 and accepts.

Another family of cases shuffle between the largest and smallest hexadecimal words, for example, `[FFFFFFFFFFFFFFFF,0000000000000000,FFFFFFFFFFFFFFFF,0000000000000000]`. This creates repeated changes between the swap and no-swap paths.

Finally, we tested lists containing only repetitions of one word. This checks that the machine handles equal comparisons across the entire list without changing its contents.

In total, the test suite executed 736 test cases. All of the test cases passed and none of the test cases failed. For every case, the final tape matched the Python sorted result, the machine accepted and the head returned to cell 0. Together with the correctness argument above, these results support that the machine behaves correctly on the specified input format.

As the only way for the machine to enter error state is when the tape does not follow the given grammar. Yet, this is not the case as the grammar is promised to be correct. Therefore, there is no test on invalid tape.

6 Conclusion

In conclusion, we designed a Turing Machine that uses bubble sort to sort a list of 16-digit hexadecimal words in ascending order. The machine performs comparison and swapping using different states and temporary markers to keep track of the processed digits. After each pass, the active part of the list is reduced by one word until the sorting is completed. The machine then restores all temporary symbols, returns the head to cell 0 and accepts.

The worst-case write complexity of the machine is shown to be $\Theta(n^2)$, where n is the tape length. Although this approach requires multiple passes through the list, it provides a straightforward way to implement sorting using a single tape without additional storage. The correctness argument also shows how the comparison, swapping and restoration operations preserve the original words while producing the sorted list.

Appendices

A. Authorship and AI Declaration

Name	**Purpose and how it was used**	**Interactions/ prompts**
ChatGPT	Ask how to do some final LaTeX typesetting, no intellectual writing is generated. Formatting is re-implemented by ourself. Formatting including: - Aligning the text and equations - Formatting table - Other formatting questions such as preambles, float, etc	`https://chatgpt.com/share/6ae73aa1-f8b8-83ec-bcec-fa965a35a759`

Beyond the declaration, we confirm that AI was not used for writing code, writing the report, or any other intellectual part tested by the assignment on behalf of each of our individual contributions detailed in the contribution appendix.

For the development of the Turing Machine, we used a Python script (`TSVgen.py`) to generate the transition table by running `python3 TSVgen.py > turing.tsv`. Detailed individual contributions to the Turing Machine are also provided in the contribution appendix.

Signed:
Luu
Minh
Thien
Date: 02 November 2026

B. Contribution

Luu Minh Thong Tran - 14522561

- Turing machine: transitions from group 6 - 9
- Test: 3 edge cases families(4, 5, 6) and complexity analysis
- Report: Proof of worst case complexity and Introduction

Minh Hoang Le - 25977873

- Turing machine: transitions group 1-5
- Test: 3 edge cases families (1, 2, 3) and documentation
- Report: Algorithm choice, and correctness of machine

Thien Bao Minh Dang - 14512584

- Test: Initialised test case generator script
- Report : Machine design and documentation, conclusion, algorithm pseudo-code, initial draft for all parts, grammar and wording improvement for all parts in report.

C. Number of steps taken in the test cases visualization

We proved that the write complexity was $\Theta(n^2)$, here, we provided a visualisation of the write complexity. The following figure shows the number of steps taken for different tape length in the test cases. It highlights a clear quadratic growth.

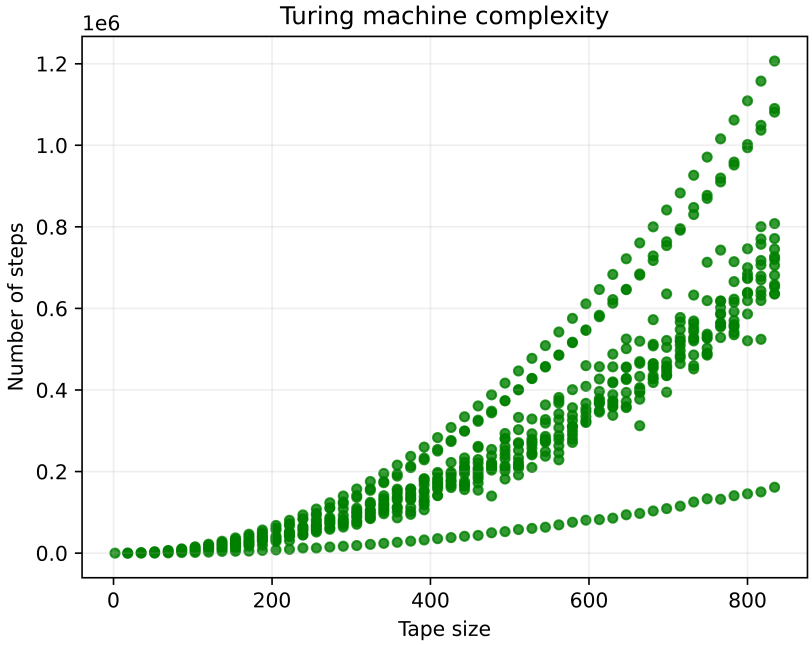


Figure 1: Complexity plot